

Last modified on

Sports Data Soccer API - Match Preview (MA8)

Get match preview data, information about previous meetings between contestants and each contestant's form in the last six matches

Overview

The Soccer **Match Preview feed** provides match preview data and contains information about previous meetings between the contestants. It also provides details about each contestant's form in their last six matches.

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 11+13	TIER 8+9+10	TIER 6+7	TIER 5	TIER 1+2+3
Updated within 7 days of a match being played - there may be a change to the content of the feed from day-to-day if a game is played in between	Once on the morning of a given match day and once within 7 days of the fixture start date/time							n/a

Match Preview (MA8)

USAGE	FEED
GET match preview data by fixture UUID (path parameter)	/soccerdata/matchpreview/{outletAuthKey},

4/10/26, 2:10 PM

Match Preview (MA8)

USAGE	FEED
GET match preview data by fixture UUID (fx parameter - supports legacy IDs)	/soccerdata/matchpreview/{outletAuthKey};

Query Parameters And Example Request Parameters

This feed also makes use of the [Global query parameters](#), and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/matchpreview` feed by a specific fixture (match) UUID.

Legacy IDs: You can use the `fx` parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

How to use query parameters and make successful requests:

- In our guides, we use HTTPS syntax for our example GET requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the [main guide](#) and [API Overview and Authorisation guide](#).
- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
 - Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
 - Use the required `_` (underscore) for any parameters listed with this prefix.
 - Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
 - If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>{fixtureUuid}</code> Fixture UUID	GET match preview data by the specified fixture UUID (path parameter method) . Pass the fixture UUID in the request URL directly as the <code>{assetId}</code> (asset/resource ID). GET https://api.performfeeds.com/soccerdata/matchpreview/{outletAuthKey}/{fixtureUuid?_rt={mode}&_fmt={dataFormat}
<code>fx</code>	Get match preview data by the specified fixture UUID (query parameter method) . Pass the fixture UUID to the <code>fx</code> parameter.

Query parameter	Value and description
	GET https://api.performfeeds.com/soccerdata/matchpreview/{outletAuthKey}?_rt={mode}&_fmt={dataFormat}&fx={fixtureUuid}

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▶<matchPreview> (XML Only)

▶<matchInfo>

▶<description>

▶<sport>

▶<ruleset>

▶<competition>

▶<country>

▶<tournamentCalendar>

▶<stage>

▶<series>

▶<contestants> (XML Only)

▶<contestant>

▶<country>

▶<venue>

▶<previousMeetings>

▶<match>

▶<contestants>

▶<goals> (XML Only)

▶<goal>

▶<previousMeetingsAnyComp>

▶<match>

▶<contestants>

▶<goals> (XML Only)

▶<goal>

▶<form>

▶<match>

▶<contestants>

▶<goals> (XML Only)

▶<goal>

▶<formAnyComp>

▶<match>

▶<contestants>

▶<goals> (XML Only)

▶<goal>